

BIOGOALS.SELECT_USER_MANUAL

User manual version: 1.0

Davide Carecci, 2025

TO DO: check on UIT machine the folders.

Preliminaries

The following user manual is intended to guide the other end-users of the *Biogoals.Select* tool. This tool is intended to be used by biomethane plant operators. It has the aim to provide a relatively simple/'conventional' way to track (*online*) a desired biomethane production value, adjusting the feeding of one co-feedstock and rejecting external disturbances while guaranteeing safety throughout the operations. It is based on feedback control theory: it employs two standard PI controllers in 'parallel' configuration, and extended for override capabilities of the integrator state when *switching* from one to the other. When coupled with the setpoints derived from a diet optimisation carried out with the aid of the *Biogoals.agri-AcoDM* tool, it is a promising solution to avoid digesters' overly conservative working conditions i.e. pushing the loading condition up to its user-defined safety limit, increasing biomethane production.

The tool was experimentally proved (twice) able to stir *online* the biomethane production from a *nominall/initial* equilibrium/operating point to a *optimal/final* operating point. It is compatible with nowadays full-scale applicability as it requires only the biogas total flow rate and volumetric composition in terms of CH_4 and CO_2 as *online*-measurable data. It is composed by a methane flow rate ($q_{M,gb}$) PI controller called *header* and a CO_2 over CH_4 biogas composition ratio ($x_{C,gb}/x_{M,gb}$ i.e. 'safety' proxy) PI controller called *follower*. When the 'safety' proxy is above a user-defined threshold, the main goal of the controller becomes the regulation of the 'safety' proxy itself (*follower* is active), whereas the biomethane production improvement is the primary goal when safe operations are in place (*header*). The outputs' setpoints y_{ref} and the optimised feeding schedule of the other co-feedstocks d_{ref} are obtained with an *offline* agri-AcoDM optimisation done before the actual *online* transition from *initial* to *optimal* operations.

In the example of the experimental validations carried out during the PhD project, the control action u was the daily load of tomato sauce converted to an *on/off* PWM duty cycle of the actuator i.e. peristaltic pump. See [IEEE_CDC2024](#) for further details.

Before actual *in plant* operation, the user can:

- run an 'open-loop' example (just to check what the code does), running 'test_closedloop_selector.ipynb' commenting the 'integration' part;
- run an 'open-loop' example (just to check what the code does), running "**UIT_selectorPI_operative.py**" commenting cells 13-14 and adding to the proper directory the data extracted from the UIT machine.

- run a 'closed-loop' example running 'test_closedloop_selector.ipynb' without commenting the 'integration' part: a proper *Biogoals.agri-AcoDM* implementation is required in this case to serve as the *real plant* simulator;
- run a 'closed-loop' example running the **bla bla** model inside *Biogoals.agri-AcoDM*.

Note: in addition to the standard Python libraries present in **requirements.txt** the custom Python library **general_utils** is needed to run these codes. How to install it:

- if the user prefers a local installation in his/her computer: download the repository from [GitHub](#), then run `pip install "path_to\general_utils"` in your Python virtual environment/Jupyter kernel;
- otherwise, install directly from GitHub running `pip install git+https://github.com/DaveCacci/general_utils.git` or `pip install git+https://<GITHUB_TOKEN>@github.com/DaveCacci/general_utils.git` if the repository is private (ask the creator for the token if not already available or deprecated).

Files description

- **"SelectorPI_pseudo_code.pdf"**: pseudo-code in PDF file as further material to understand the theory and the actual implementation of the controller.
- **"SelectorPI_controller.py"**: Python file that contains the actual '*PIcontroller*' and '*HysteresisComparator*' classes (the two main blocks shown in the '*Closed-loop scheme*' section of this manual). Both the version implemented on the UIT reactor (i.e. old, Cremona experimentation 2023-2024) and the one on the BioTA lab reactors (Chile experimentation 2025) are present inside.
- **"UIT_implementation/UIT_software_user_manual.pdf"**: user manual of the proprietary software of the UIT test facility.
- (MAIN) **"UIT_implementation/UIT_selectorPI_operative.py"**: main controller code, program, Python file.
- **"UIT_implementation/UIT_results_reader.ipynb"**: Jupyter notebook to read, analyse and plot the outputs of `"/UIT_selectorPI_operative.py"`.
- **"UIT_implementation/setpoint{date}.csv"**: CSV input data file containing the setpoints.
- **"UIT_implementation/Bioreactor/"**: sub-folder that contains the configuration INI files read by the UIT proprietary software on the UIT machine.
- (MAIN) **"RPI_implementation/RPI_selectorPI_operative.py"**: main controller code, program, Python file.
- **"RPI_implementation/test_closedloop.ipynb"**: Jupyter notebook to run `"/RPI_selectorPI_operative.py"` from.
- **"RPI_implementation/Input/y_df_data_on_alt_R1.csv"**: CSV input data file containing the *online* measurements already preprocessed and ready to be read for the computation of control action purposes.
- **"RPI_implementation/Input/Output_setpoints.txt"**: TXT input data file containing the setpoints. Same for ".csv".

- **"RPI_implementation/Input/SELECTOR_pwm_actual.csv"**: CSV file storing the control action values in terms of parameters of the duty cycle (to be read by the actuator).
- **"RPI_implementation/Output/Boundary_calc_conditions.csv"**: CSV output file containing measured data and setpoints' values at each control step.
- **"RPIimplementation/Output/State{header/follower}_{date}.csv"**: CSV output file storing the quantities computed and required to run the *header* and *follower* PI controllers (e.g. activeness boolean condition, past integral terms, feedback errors etc.).
- **"RPIimplementation/Output/Output_selector{date}.csv"**: CSV output file storing the feedback errors and actual control actions computed by the 'selectorPI' at each control step.

The program functioning is based on the use of the *pandas* Python library together with *Timestamps* i.e. objects created with the *datetime* library that index some other quantities with date and time values (e.g. 2025-10-24 13:25:00).

When implemented on real reactors (not in simulation with Biogoals.agri-AcoDM as real plant) the main difference between the **"RPI_selectorPI_operative.py"** and the **"UIT_selectorPI_operative.py"** is that the former reads already pre-processed data (by **bla bla**), whereas data pre-processing is performed inside **"UIT_selectorPI_operative.py"** (it was the only Python file needed to run the controller on the UIT reactor). As a result, by now, **"RPI_selectorPI_operative.py"** can be run only at times equal to rounded hours (e.g. HH:00:00, not at HH:MM:00 with $MM \neq 00$), whereas **"UIT_selectorPI_operative.py"** can be run at any time.

"SelectorPI_controller.py"

This code is a **collection of declarations of Python classes** and their methods (i.e. functions inside classes), so it is meant to be imported in other codes/notebooks, not to be run by its own. Some standard libraries are imported. The main difference between the classes used during the experimentation in Chile with RPi and the `'_uit'` classes is that the former were defined with the help of the **@dataclass** standard Python library and has a *logger* attribute that is an instantiation of the **logging** standard Python library (a better way to log intermediate info, warnings and errors when running codes online like in the case of a closed-loop controller). Since the main methods are practically the same or does the same operations, only the RPi formulation is going to be discussed here more in details (however, the code is overall well commented).

- The **PIcontroller** class: **if a Pcontroller only is desired**, as it was for the experimental validations, the user must **COMMENT MANUALLY THE INTEGRAL ACTION AT LINE 57 and 236**. It contains the functions to compute a new control action, load the past integral state, override the integral state with another value (e.g. the one of the other parallel controller in the 'selector' scheme) and save out to a *pandas dataframe* the updated computed quantities.

- The **HysteresisComparator** class: it compares the value of the 'safety' proxy with user-defined thresholds ($threshold_low < threshold_high$ to avoid chattering behaviours of the controller). It has a boolean state (True or False) that is changed if the input value to this function is above the $threshold_high$ or below the $threshold_low$ (remains the same if within the two). **Actually, it was modified to have change of state also if I go below $threshold_high$...not a problem when there are discrete data of biogas analysis...but in theory to be corrected!!** .

"UIT_selectorPI_operative.py"

Standard libraries are imported as well as some custom functions present in a different or the same directory and, as mentioned above, some functions from the **general_utils** library. If a function present in another directory is needed, add `sys.path.insert(0, 'your_directory')` (e.g. 'SelectorPI_controller.py' in the example provided). Note that on the UIT machine present in the "Fabbrica della Bioenergia" A. Rozzi laboratory (Politecnico di Milano, Cremona campus), Windows 7 is installed, so that Python 3.8 is the last version supported (there are no major compatibility issues for the present code to run with such interpreter, but for the Pandas "*append*" commands that must be replaced with "*_append*"). See Chapter 6 of 'UIT_software_user_manual.pdf' for further information on where to place this file to be correctly integrated with the UIT machine.

- The data from the UIT machine are saved in three different sub-folders ('R1', 'R2' and 'Analyse'). The data in, for example, the 'R1' sub-folder are stored in CSV files with the date as filename in the format 'YYYY-MM-DD_R1.csv'. The user **SPECIFIES THE REACTOR NAME, THE DIRECTORY AND THE AMOUNTS OF DAYS OF INPUT DATA FILES TO BE READ**. Two days (*today* and *yesterday*) are read in the example (minimum recommended) and concatenated in one unique Pandas *dataframe*.
- **Data preprocessing**: the user **SETS THE THRESHOLD** for outlier removal as an the maximum distance the Z-score of a data point can be from 0 in terms of integer-times the standard deviation of data. Data points characterised by a Z-score $> threshold$ are substituted with a linear interpolation between the previous and the next 'non-outlier' data points (done on both biogas flow rate and CH_4 and CO_2 fractions). CH_4 flow rate and biogas CO_2/CH_4 ratio are computed. The user **SETS THE WINDOW SIZE** for the moving average of flow rates (in particular needed if the reactor mixing is carried out with on-off duty cycles). Data are extracted at the current control step *Timestamp* (last row of data).
- **Plot** the data (original signal vs outlier removed vs filtered with moving average).
- The user **SPECIFIES THE DIRECTORY OF THE INPUT DATA FILE** that contains the setpoints (Time or *Timestamp*, methane flow rate, biogas composition ratio and relative lower and higher thresholds). It can be in TXT or CSV format depending on how it is created. If CSV, as in the example, no need for further manipulation is needed. For logging purposes, the user **SPECIFIES THE DIRECTORY FOR AN OUTPUT CSV FILE** where all the quantities of interest for the current control step are saved to.

- The user **SETS THE CONTROL DESIGN PARAMETERS** (control interval dt in seconds, *saturation_high* and *saturation_low* in $\text{m}^3 \text{s}^{-1}$, *kp*, *Ti* in seconds). The controllers are instantiated with the **PIcontroller** class. The last-available states of the controllers' integrator part are loaded with the aid of the **load_state** method (the user **SPECIFIES THE FILENAMES AND DIRECTORY FOR AN OUTPUT CSV FILE** for both controllers).
- **Override**: extract the previous boolean conditions of *selection* for both controllers (which was the "active" one between the two). Update the *selection* based on the **HysteresisComparator** class (that compares the measured biogas composition ratio with thresholds) and its **update** method. If *selection* has changed its value at the current run, compute the value needed to override the integrator of the controller that is turning from inactive to active/selected to be consistent with the the last-available *control_signal*. Then, override with the aid of the **reset_state** method.
- **Compute** the control action for the current control step of both controllers and select the one of the controller that is active/selected for the current control step (if the measured biogas composition ratio is higher then the user-defined safety threshold, select the *control_signal* of the 'follower' controller). Save the updated controllers' states with the aid of the **save_state** method. Note that the output of the **compute** method is in $\text{m}^3 \text{s}^{-1}$ and it must be converted to mL d^{-1} .
- **Convert** the control action from a continuous *float* to a PWM on-off duty cycle couple (*on_minutes_ini*, *off_minutes_ini*). To do so, the on-time seconds of one period are fixed to 60 seconds (1 minute). Add the nominal/initial value of the control action (i.e. *on_minutes_nominal*) to the *control_signal* to obtain the actual value of the control action to be communicated to the actuation system (see Section *Additional notes* for details). The user **SPECIFIES THE CHARACTERISTIC FLOW RATE OF THE PUMP**. The user **SPECIFIES THE FILENAME AND DIRECTORY FOR AN OUTPUT CSV FILE** where all the quantities of interest computed in the current control step are saved to.
- The latest-available **configuration INI file** of the UIT proprietary software is loaded and **updated** with the above-mentioned PWM values (*on_seconds_ini*, *off_seconds_ini*). The user **SPECIFIES INPUT, OUTPUT AND COPY PATHS OF THE CONFIGURATION FILES**.
- The user **SPECIFIES THE DIRECTORY TO WRITE LOG** with all success, warning and error messages.

"RPi_selectorPI_operative.py"

Standard libraries are imported as well as some custom functions present in a different or the same directory and, as mentioned above, some functions from the **general_utils** library. If a function present in another directory is needed, add `sys.path.insert(0, 'your_directory')` (e.g. 'SelectorPI_controller.py' in the example provided). Afterwards, the **main()** function is declared to allow the script to be executed from the terminal (with command-line arguments) and other scripts (nominally, this code is imported into **bla bla** to be run. The advice when performing simulations is too have both "RPi_selectorPI_operative.py" and **bla bla** open in your editor, to modify this code and then

testing it. If the user wants to run the present code as stand alone, delete **main()** function definition line and unindent the code below). For the reading and saving process of different simulation scenario avoiding the override of old results and confusion, the user **SETS A *modelname*** string to be used as **main()** argument. Is a specific folder is desired for the input and output files of the current simulation, **SET A *testname*** equal to the name of the desired subfolder to the specified *directory* (where this code is placed is default). The **logger** of the info (very comfortable and smart when running this code online) is setup **SETTING LOGS SUBFOLDER AND FILE NAMES**. Afterwards:

- The current control step *Timestamp* is defined as the current time (i.e. =*end_timestamp*) and has to be rounded to the nearest hour (HH:00:00) only if this program is coupled with *bla bla*. The user **SETS THE NUMBER OF DAYS TO FILTER THE INPUT DATA CSV** in the definition of *start_timestamp*.
- The user **SPECIFIES THE DIRECTORY OF THE INPUT CSV DATA FILE** that contains the *online* measurements (*Timestamp*, biogas rate '*gas_rate_out_ma*' (after outlier removal and moving average), biogas CH_4 '*xM_gb_out*' CO_2 '*xC_gb_out*' and fractions). In the example, *gas_rate_out_ma* must be in $L\ h^{-1}$, whereas *xM_gb_out* and *xC_gb_out* as fraction $\in [0, 1]$. The user **SETS THE NUMBER OF DAYS TO FILTER THE INPUT DATA CSV** in the definition of *start_timestamp*. CH_4 and CO_2 flow rates, and biogas CO_2/CH_4 ratio are computed. Data are extracted at the current control step *Timestamp* (last row of data).
- The user **SPECIFIES THE DIRECTORY OF THE INPUT DATA FILE** that contains the setpoints (Time or *Timestamp*, methane flow rate, biogas composition ratio and relative lower and higher thresholds). It can be in TXT or CSV format depending on how it is created. If TXT, as in the example, a dataframe must be created parsing Time to *Timestamp* values, specifying the *datetime* correspondent to the first Time value). For logging purposes, the user **SPECIFIES THE DIRECTORY FOR AN OUTPUT CSV FILE** where all the quantities of interest for the current control step are saved to.
- The **SETTING OF THE CONTROL PARAMETERS**, the **controllers' instantiation**, integrator state load, and the **override** and **compute** part are almost equal to what have been already described for "UIT_selectorPI_operative.py". Note: add the nominal/initial value of the control action to the *control_signal* to obtain the actual value of the control action to be communicated to the actuation system (see Section *Additional notes* for details).
- **Convert** the control action from a continuous *float* to a PWM on-off duty cycle couple (*tuple_seconds*). To do so, the user **SPECIFIES THE ON-TIME SECONDS OF ONE WAVE PERIOD AND THE CHARACTERISTIC FLOW RATE OF THE PUMP**. The user **SPECIFIES THE DIRECTORY FOR AN OUTPUT FILE** where the PWM signal is saved. The user **SPECIFIES THE FILENAME AND DIRECTORY FOR AN OUTPUT CSV FILE** where all the quantities of interest computed in the current control step are saved to.
- The **FlexiblePlotter class** is used to **plot** the main outputs of interest from *start_timestamp* to *now*. The user can select which data to plot and the plotting parameters of the **FlexiblePlotter** instantiation following the guidelines provided in its

documentation (see...). The user SPECIFIES THE FILENAME AND DIRECTORY FOR THE OUTPUT PNG FIGURE.

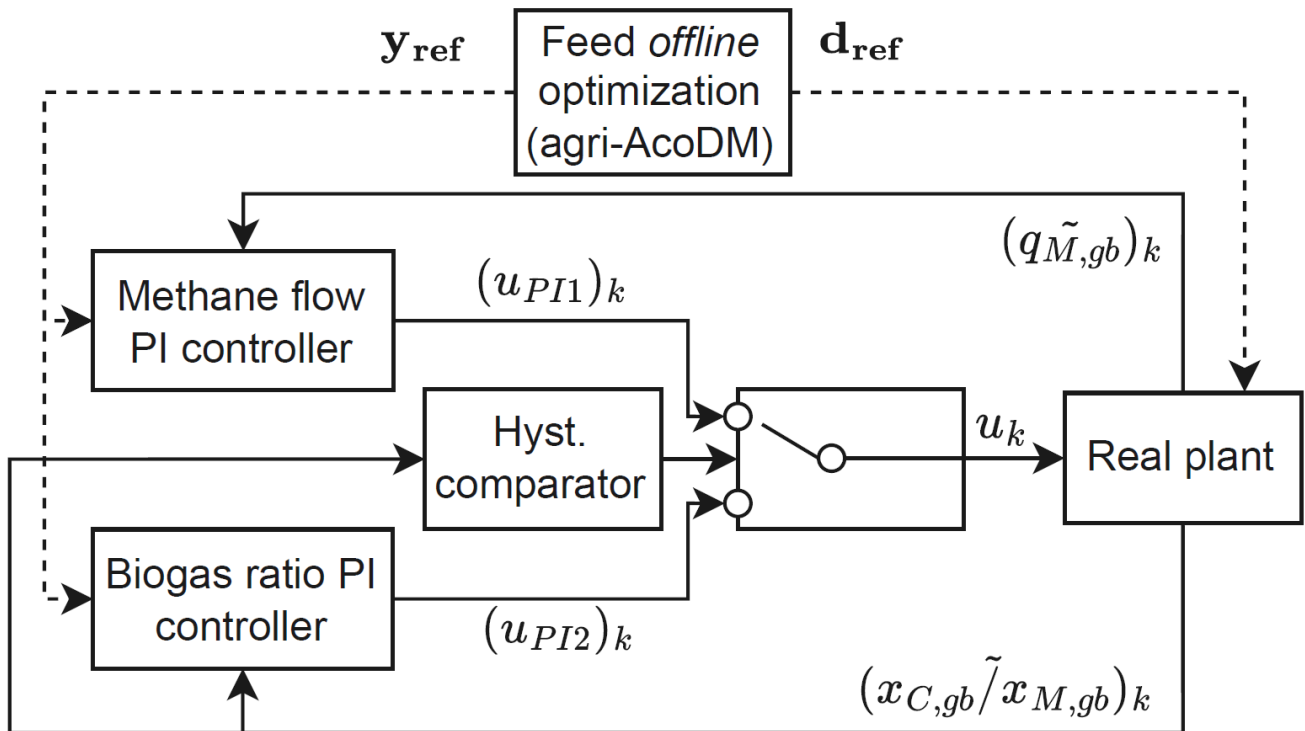
- The last unindent code lines follow the **main()** declaration for it to be executed from the terminal (with command-line arguments) and other scripts (e.g. bla bla).

Additional notes

If the input data file for setpoints is in TXT form, it means it is exactly the same file used as CombiTimeTable for the agri-AcoDM model (see...).

If the input data file for setpoints is in CSV form, it means it was created with ...

Closed-loop scheme



where k is the index of the current control step (in practice, the *Timestamp* in which the program is run).

Note that this control scheme was and shall be tuned around the *nominal/initial* equilibrium/operating point: the control action computed by the **PIcontroller** class shall be null when working around such point. The control action required to sustain such working point (i.e. the *nominal_u*) must be then added to the *control_signal* returned by the *compute* method of the **PIcontroller** class. In the scheme above, u_{PI1} is the control action's correction/deviation computed by the *header*, whereas u_{PI2} is the one computed by the *follower*. One of the two is selected based on the boolean information provided by the **HysteresisComparator** block, and finally the nominal/initial value of the control action (u_0 in "**SelectorPI_pseudo_code.pdf**") is added to obtain the actual value to be given to the PWM converter function.